

توضیحات کد

قسمت دوم

در این بخش باید عکس را 180 درجه میچرخانیم تا به حالت طبیعی برسد، برای این کار از سه متود استفاده کردم که به اختصار به آنها پرداخته میشود

متود اول: `img[::-1,::-1]`

در این روش ترتیب عناصر لایه های اول و دوم برعکس میشود که باعث بازتاب عکس در دو راستای افقی و عمودی است. نتیجه ان تقارن 180 درجه ای عکس خواهد بود.

```
In [ ]: # 1st way
rotated_img1 = img[::-1,::-1]
show_image(rotated_img1)
```

متود دوم `np.flip()`:

در این روش با دادن دو پارامتر به این تابع، ارایه مورد نظر را نسبت به محور مشخص شده (پارامتر دوم) بازتاب میدهد. در اینجا ابتدا عکس را بصورت عمودی بازتاب میدهد `axis=0` و زمانی که `axis=1` باشد عکس را نسبت به محور افقی بازتاب میدهد.

```
In [ ]: # 2nd way
flipped_img = np.flip(img, 0)
# show_image(flipped_img)
rotated_img2 = np.flip(flipped_img, 1)
show_image(rotated_img2)
```

متود سوم: `np.rot90()`

در این روش با استفاده از این تابع، عکس را دو بار به اندازه 90 درجه دوران میدهد. پارامتر اول ان ارایه است و پارامترهای دوم و سوم ان به ترتیب تعداد دفعات دوران عکس و محور دوران را مشخص میکند

```
In [ ]: # 3rd way
rotated_img3 = np.rot90(img, 2, (0,1))
show_image(rotated_img3)
```

قسمت سوم

در این قسمت هیستوگرام مقادیر پیکسل های هر چنل رنگی را به کمک تابع `plt.hist()` به نمایش گذاشتم

رنگ هیستوگرام را تعیین میکند `color`:

رنگ لبه میله های نمودار را مشخص میکند. `ec`:

شفاف بودن میله های نمودار را تعیین میکند. `alpha`:

```
In [ ]: plt.figure(figsize=[20,40])
plt.subplot(311); plt.hist(img[:, :, 0].flatten(), 256, color = 'blue', ec = 'black',
plt.subplot(312); plt.hist(img[:, :, 1].flatten(), 256, color = 'green', ec = 'black',
plt.subplot(313); plt.hist(img[:, :, 2].flatten(), 256, color = 'red', ec = 'black',
```

در بخش بعدی، بزرگترین مقدار میان چنل های رنگی هر پیکسل را بدست آورده و عکس سیاه-سفید آن را به نمایش میگذارد

با استفاده از این تابع نامپای، بیشترین مقدار ارایه در هر محور را بدست می آورد.

```
In [ ]: gray_img = np.amax(img[:, :, ], axis = 2)
        show_image(gray_img)
```

قسمت چهارم

ابتدا بدلیل استفاده ای که جلوتر خواهیم داشت، هر سه چنل عکس را به کمک تابع

`ndarray.flatten()` به صورت یک بعدی در می آورد.

```
In [ ]: img_flat0 = img[:, :, 0].flatten()
        img_flat1 = img[:, :, 1].flatten()
        img_flat2 = img[:, :, 2].flatten()
```

`np.bincount()`: این تابع، به عنوان ورودی ارایه ای یک بعدی دریافت میکند که `dtype=int` باید باشد. در خروجی ارایه ای میدهد که تعداد فراوانی مقدار `m` را در ایندکس `m` این ارایه به نمایش میگذارد. به عنوان مثال اگر در ارایه ورودی 5 بار عدد 3 تکرار شده باشد، در ایندکس 3 آن عدد 5 قرار میگیرد. مثال بیشتر:

```
In [1]: import numpy as np
        np.bincount(np.arange(5))
```

```
Out[1]: array([1, 1, 1, 1, 1], dtype=int64)
```

```
In [2]: np.bincount(np.array([0, 1, 1, 3, 2, 1, 7]))
```

```
Out[2]: array([1, 3, 1, 1, 0, 0, 0, 1], dtype=int64)
```

حال در این بخش از کد، با استفاده از تابع ذکر شده، ابتدا خروجی یک چنل عکس را دریافت میکند؛ سپس لیست را بصورت نزولی به کمک `np.sort()` مرتب میکند. در نهایت طبق خواسته مسئله 5 پیکسلی که بیشترین فراوانی را دارد بدست میآورد.

```
In [ ]: m = np.bincount(img_flat0)
        n = np.sort(m)
        max_value = n[-5]
        output = m - max_value
        np.where(output >= 0)
```

قسمت پنجم

تابع `neighbors_mean`:

این تابع میانگین پیکسل های اطراف پیکسل های داده شده را محاسبه میکند و خروجی را بصورت یک لیست از مقادیر میانگین میدهد. در ابتدا با استفاده از `np.pad()` استفاده شده تا برای محاسبه میانگین نقاط لبه عکس به مشکل نخورد.

```
In [ ]: def neighbors_mean(matrix, rows, columns, color_chanel):
        pad_matrix = np.pad(matrix, (1, ), 'constant', constant_values = 0)
        mean_list = []
        for i in range(len(rows)):
            Sum = np.sum(matrix[rows[i]-1:rows[i]+2, columns[i]-1: columns[i]+2, color_
```

```
mean = (Sum - 255)//8
mean_list.append(mean)

return mean_list
```

در این قسمت با کمک `np.where()` ایندکس نقاطی که هر چنل ان برابر 255 بوده را برمیگرداند

```
In [ ]: rows, columns, color_chanel = np.where(img == 255)
```

در این قسمت از کد ابتدا تابع را صدا میزنیم و مقادیر داخل لیست را داخل تابع `iter()` میریزد. در قسمت بعدی با استفاد از `np.where` مقادیر بدست آمده را جایگزین میکند.

```
In [ ]: my_list = iter(neighbors_mean(img, rows, columns, color_chanel))
output = np.where(img == 255, next(my_list), img)
```